

```
sess.run(init_op)
print( "Slice 2:",sess.run(tf.slice(t, [1, 0, 0], [1, 2, 3])), "\n")

with tf.Session() as sess:
    sess.run(init_op)
    print( "Slice 2:",sess.run(tf.slice(t, [1, 0, 0], [2, 1, 3])), "\n")
Conclusion
Output:
Slice 1: [[[3 3 3]]]
Slice 2: [[[3 3 3]
          [4 4 4]]]
Slice 2: [[[3 3 3]]
          [[5 5 5]]]
```

Placeholder

A placeholder is simply a variable that we will assign data at a later state. It allows us to create our operations and build our computation graphs, without needing the data. In TensorFlow terminology, we then feed data into the graph through these placeholders.

```
import tensorflow as tf
x = tf.placeholder("float", None)
y = x + 2.5
with tf.Session() as session:
    result = session.run(y, feed_dict={x: [1, 2, 3]})
    print(result)
```

Output:

```
[3.5 4.5 5.5]
```

The first line creates a placeholder called `x`. It reserves a place in memory to store values at a later stage. This is used only to define the structure in the memory and no initial value is assigned into that location. Then a tensor call is used for adding `x` with 2.5. At a later stage, one can use this storage space and the defined operation to perform a vector addition. For this it is necessary to initiate a session and execute the defined operation within the session for a set of values of `x`.

Running `y` requires values of `x`. This can be assigned to run inside the `feed_dict` argument. Here, values of `x` are [1, 2, 3]. The run of the session of `y` will produce [3.5 4.5 5.5].

A larger graph consisting of numerous computations can be divided into small segments of atomic operations, and each of these operations can be performed individually. Such partial executions of a larger graph operation are an exclusive feature of TensorFlow and are not available in many other libraries that do similar jobs. During operation, placeholders do not require any static shape and can hold values of any length.

```
import tensorflow as tf
x = tf.placeholder("float", None)
y = x + 2.5
with tf.Session() as session:
    result = session.run(y, feed_dict={x: [1, 2, 3, 4]})
    print(result)
```

This script will run for four values of vector `x` and will produce: [3.5 4.5 5.5 6.5].

To perform the same operation of adding 2.5 with an array of three columns with no upper boundary for the number of rows, one can modify the above script as follows:

```
x = tf.placeholder("float", [None, 3])
y = x + 5

with tf.Session() as session:
    x_data = [[2, 3, 1],
              [4, 7, 6],]
    result = session.run(y, feed_dict={x: x_data})
    print(result)
```

Output:

```
[[ 7.  8.  6.]
 [ 9. 12. 11.]]
```

The placeholder can be extended to an arbitrary number of `None` dimensions. For example, to load an RGB image, a placeholder needs to have three slices to store all the three RGB image planes. Therefore, the placeholder needs two `None` for the first two dimensions and three for the last dimension. Then TensorFlow's slice method can be used to take a sub-segment out of the image.

```
import matplotlib.image as mpimg
import matplotlib.pyplot as plt
import os

# First, load the image again
dir_path = os.path.dirname(os.path.realpath(__file__))

filename = dir_path + "\MarshOrchid.jpg"

print(dir_path)

raw_image_data = mpimg.imread(filename)
print(raw_image_data.shape)
plt.imshow(raw_image_data)
plt.show()
image = tf.placeholder("int32", [None, None, 3])
slice = tf.slice(image, [0, 0, 0], [300, -1, 1])

with tf.Session() as session:
```